Antivirus Detection & Evasion Research Report

1. Executive Summary

Modern endpoint security solutions evaluate software threats across two primary paradigms: static signature matching and dynamic behavioral heuristics. As part of Phase 4 of the Endpoint Security Research Project, an uncompiled input-monitoring script (mainfile.py) was analyzed across multi-engine scanning infrastructure (VirusTotal) and on a native Windows endpoint using Microsoft Defender.

The primary finding of this evaluation is the operational contrast between basic multi-engine cloud signatures and local consumer endpoint protections:

- **Multi-Engine Static Scanning:** 3 out of 61 vendors flagged the uncompiled source file, identifying it specifically as Py/Keylog-ATZ.
- **Local Endpoint Scanning:** Microsoft Defender's static engine reported 0 threats, demonstrating that plain-text script files executed via trusted runtimes (e.g., python.exe) routinely bypass consumer on-disk signature heuristics unless behavioral interception is actively triggered.

2. Static Signature Analysis

Static scanning involves inspecting files at rest without execution. Scanners evaluate cryptographic hashes, byte patterns, known text indicators, and abstract syntax tree structures.

2.1 Cryptographic & File Metadata

Property	Value / Hash
File Name	mainfile.py
File Size	1.36 KB (1,394 bytes)
MD5 Hash	9bf3317714849f74077175659932f8f5
SHA-1 Hash	c0866991c769bcc837f2f847a7138f95ac9a12f4

Property	Value / Hash
SHA-256 Hash	b5a91eb0681b34d9b770478fb443b8bcadaedb136ae83ad1e085b64444ea427b
SSDEEP	24:BVomXq260Ydglo2VUWf3p1MRoPp8BALQaBDT3eEVj1z:Bqma260YdI7U+wRTA8aBDzeE912
Magic / MIME	Python script, ASCII text executable, CRLF terminators

2.2 Multi-Engine Scan Results (VirusTotal)

The static scan yielded a detection ratio of **3/61** (4.9% detection rate). The vendors detecting the file and their respective classifications are summarized below:

Security Vendor	Result / Signature	Classification Category
Sophos	Py/Keylog-ATZ	Input Capture / Spyware heuristic
ZoneAlarm by Check Point	Py/Keylog-ATZ	Input Capture / Spyware heuristic
Google	Detected	Generic Cloud Signature
Microsoft Defender (Cloud)	Undetected	Clean
CrowdStrike Falcon	Undetected	Clean
SentinelOne (Static ML)	Undetected	Clean
Kaspersky / BitDefender	Undetected	Clean

2.3 Local Windows Defender Verification

A dedicated custom scan was executed locally on Windows Security against mainfile.py. The scan executed in 1 second over 1 file and returned **0 threats found**. This verifies that standard consumer signatures do not statically flag the script on disk, primarily because plain Python scripts rely on the system interpreter rather than native Portable Executable (PE) headers.

3. Behavioral & Heuristic Evaluation

Dynamic behavioral analysis examines software actions during live execution within an instrumented sandbox or monitored endpoint environment.

3.1 Sandbox Execution Telemetry (CAPE Linux)

- **Process Hierarchy:** Execution occurred under process tree PID 2284 (xterm -hold -e /usr/bin/python /tmp/mainfile.py) spawning child process PID 2285 (/usr/bin/python /tmp/mainfile.py).
- **File System Activity:** The script interacted with terminal and pseudo-terminal file descriptors:
    - /dev/ptmx and /dev/pts/0 (Read/Write)
    - /dev/tty (Read/Write)
    - System library handles: /lib/x86_64-linux-gnu/libcrypto.so.3
- **Network Telemetry:** Egress traffic was recorded via DNS resolutions toward api.snapcraft.io over TCP port 443 (185.125.188.54:443, JA3 Digest: 473cd7cb9faa642487833865d516e578).

3.2 Windows Endpoint Detection & Response (EDR) Architecture

In enterprise Windows environments, EDR systems monitor specific user-mode behaviors to detect unauthorized input capture:

- **User-Mode API Hooking (SetWindowsHookEx):** User-mode libraries like pynput invoke the Win32 API SetWindowsHookExW with parameter WH_KEYBOARD_LL (13) to intercept global keyboard events. EDR sensors monitor API invocations across thread boundaries, flagging background processes that install global low-level hooks without owning an active window handle or GUI focus.
- **Persistence via Startup Registry Keys:** Unauthorized scripts frequently seek persistence by inserting values under HKEY_CURRENT_USER\Software\Microsoft\Windows\CurrentVersion\Run. Modern EDR frameworks use kernel-level callbacks (e.g., via CmRegisterCallbackEx) to identify unauthorized registry write events initiated by non-administrative user processes.

4. Theoretical Evasion Analysis (Literature Review)

In cybersecurity research, understanding how malicious actors attempt to bypass static and heuristic detections informs better defensive architecture.

Technique	Theoretical Evasion Mechanism	Defensive Countermeasure
Code Obfuscation (e.g., PyArmor)	Serializes code objects using Python marshal structures and obfuscates bytecode and abstract syntax trees (AST). This breaks literal pattern matching and hash-based signatures.	Runtime memory dumping (e.g., hooking PyMarshal_ReadObjectFromString) and heuristic scoring of packed/high-entropy script files.
String Encryption	Encrypts literal strings (such as file paths, DLL names, and registry keys) within the source code, decrypting them in memory only at runtime to evade static pattern scrapers.	Behavioral tracing of system call arguments and dynamic API parameter inspection regardless of how strings are stored on disk.
In-Memory Execution	Loads payloads directly into memory buffers via dynamic reflection or process hollowing, preventing malicious binary artifacts from ever touching the disk filesystem.	Memory scanning, Antimalware Scan Interface (AMSI) script inspection, and tracking anomalous memory regions marked with Read-Write-Execute (RWX) permissions.

5. Conclusion & Recommendations

The experimental results confirm that static file signatures alone are insufficient to defend against script-based threats, as evidenced by the clean verdict from Windows Defender and 58 out of 61 VirusTotal engines. Effective endpoint security requires a layered defense:

1. Enforcing application control and execution policies (such as AppLocker or WDAC) to restrict unauthorized script execution.
2. Deploying behavioral EDR sensors capable of detecting low-level input hooks (WH_KEYBOARD_LL) and unverified registry modifications.
3. Implementing Multi-Factor Authentication (MFA) to mitigate the impact of stolen credentials.